1. Overview of the page

The uploaded PDF is an ACE-oriented tutorial about** ** **Cloud Run: deploying containerized applications** . Its main purpose is to teach you how to deploy a container to Cloud Run, configure traffic, manage revisions, control access, connect to other Google Cloud services, use secrets, trigger services from events, and distinguish Cloud Run Services from Cloud Run Jobs. The PDF explicitly says these topics are exam-relevant for the Associate Cloud Engineer exam.

Cloud Run itself is Google Cloud's** ** **fully managed serverless platform for running containers, services, functions, jobs, and event-driven workloads** . Google's current documentation describes Cloud Run as a fully managed application platform that abstracts infrastructure management and runs workloads on Google's scalable infrastructure. (Google Cloud Documentation)

You should learn five big ideas from this document:

1. **How Cloud Run runs containers without you managing servers.**
2. **What your container must do to be accepted by Cloud Run.**
3. **How deployments create immutable revisions.**
4. **How traffic, IAM, scaling, networking, secrets, and events work.**
5. **When to choose Cloud Run Service versus Cloud Run Job.**

---

2. Glossary of new terms

Term	Definition	Why it matters
Cloud Run	Google Cloud's fully managed platform for running containers, services, functions, jobs, and event-driven workloads.	It is the central service in this material.
Serverless	A cloud model where you deploy code or containers without managing servers directly.	You focus on the app, not VM provisioning, patching, or scaling.
Fully managed	Google manages the underlying infrastructure, scaling, availability, and runtime platform.	This is why Cloud Run is simpler than manually running containers on Compute Engine or GKE.
Container	A packaged application with its runtime, dependencies, and execution environment.	Cloud Run runs containers.
Container image	A versioned, deployable package used to create running containers.	Cloud Run deploys from an image stored in a registry.

Term	Definition	Why it matters
Artifact Registry	Google Cloud service for storing container images and other artifacts.	Your Cloud Run service usually pulls its image from Artifact Registry.
Stateless	The application does not depend on local persistent state between requests.	Cloud Run instances can be stopped, restarted, or scaled to zero.
Instance	A running copy of your container created by Cloud Run.	Scaling means Cloud Run adds or removes instances.
Scale to zero	Cloud Run can reduce running instances to zero when there is no traffic.	Saves money but can introduce cold starts.
Cold start	Delay when Cloud Run must start a new container instance before serving a request.	Important for latency-sensitive apps and exam questions.
`min-instances`	Minimum number of Cloud Run instances kept warm.	Setting it to `1` or more reduces or avoids cold starts.
`max-instances`	Maximum number of instances Cloud Run may create.	Protects databases or downstream systems from overload.

Term	Definition	Why it matters
Concurrency	Number of simultaneous requests one Cloud Run instance can handle.	Controls how many requests are packed into each instance.
Request timeout	Maximum time a request can run before Cloud Run terminates it.	Important for long-running HTTP requests.
HTTPS endpoint	Public or controlled URL exposed by Cloud Run for a service.	Cloud Run automatically gives services a reachable endpoint.
`run.app` domain	Default Cloud Run service domain.	Cloud Run services receive stable service URLs.
HTTP/1	Classic HTTP protocol version.	Supported by Cloud Run.
HTTP/2	Newer HTTP protocol version with multiplexing and better efficiency.	Supported by Cloud Run.
gRPC	RPC framework commonly using HTTP/2.	Cloud Run can host gRPC services.
WebSocket	Persistent bidirectional connection between client and server.	Cloud Run supports WebSocket-style workloads.

Term	Definition	Why it matters
Container contract	Requirements your container must satisfy to run correctly on Cloud Run.	If the contract is broken, deployment or serving fails.
`PORT` environment variable	Environment variable telling your container which port to listen on.	Cloud Run expects your app to listen on this port.
`0.0.0.0`	Network bind address meaning “listen on all interfaces.”	Required so Cloud Run can route traffic into the container.
`127.0.0.1`	Loopback address meaning “only inside the container itself.”	Wrong for Cloud Run HTTP serving because external traffic cannot reach it.
Ephemeral disk	Temporary local filesystem that does not persist reliably across instances.	You should not store durable files inside the Cloud Run container.
Deployment	Process of creating or updating a Cloud Run service or job.	Each service deployment creates a new revision.
Revision	Immutable snapshot of a Cloud Run service configuration and container image.	Enables rollbacks, canaries, and traffic splitting.

Term	Definition	Why it matters
Immutable	Cannot be changed after creation.	A revision is not edited; instead, a new revision is created.
Traffic management	Routing percentages of requests to one or more revisions.	Used for rollbacks and canary deployments.
Canary deployment	Sending a small percentage of traffic to a new version before full rollout.	Reduces deployment risk.
Rollback	Sending traffic back to a previous known-good revision.	Used when a new revision has bugs.
Revision tag	Named URL target for a revision.	Lets you test a revision directly without giving it normal production traffic.
IAM	Identity and Access Management.	Controls who can deploy, manage, or invoke Cloud Run.
`run.invoker`	IAM role allowing a principal to call a Cloud Run service.	Required for authenticated access.
`run.developer`	IAM role for deploying and managing Cloud Run resources.	Useful for developers who should not have full admin control.
`run.admin`	IAM role with broad Cloud Run control.	Powerful role; use carefully.

Term	Definition	Why it matters
`allUsers`	IAM principal meaning anyone on the internet.	Used when making a Cloud Run service public.
Identity token	Token proving caller identity to an authenticated Cloud Run service.	Required when calling private Cloud Run services.
Service account	Google Cloud identity used by services and workloads.	Cloud Run uses a runtime service account to access other GCP services.
Ingress	Controls what network paths may reach the Cloud Run service.	Determines whether traffic can come from internet, internal sources, or load balancers.
Egress	Outbound traffic from Cloud Run to other systems.	Important for reaching private databases, APIs, or VPC resources.
VPC	Virtual Private Cloud network in Google Cloud.	Private resources often live inside a VPC.
Direct VPC egress	Newer recommended method for Cloud Run to send traffic to a VPC without a connector.	Google documentation identifies it as the recommended VPC egress option. ([Google Cloud Documentation](https://docs.cloud.google.com/run/docs/configuring/vpc-direct-vpc?utm_source=chatgpt.com "Direct VPC with a VPC network
Serverless VPC Access connector	Older/alternative bridge from serverless products to VPC resources.	Used when Cloud Run must call private IP resources.

Term	Definition	Why it matters
Secret Manager	Google Cloud service for storing secrets securely.	Used to inject passwords, tokens, and credentials into Cloud Run.
Environment variable	Key-value configuration injected into the process environment.	Common way to pass config into Cloud Run containers.
Secret mount	Mounting a secret as a file inside the container.	Useful when apps expect credentials as files.
Pub/Sub	Google Cloud messaging service.	Can push messages to Cloud Run over HTTP.
Push subscription	Pub/Sub subscription that sends messages to an HTTP endpoint.	Lets Pub/Sub invoke Cloud Run automatically.
Eventarc	Google Cloud event routing service.	Recommended pattern for wiring Google Cloud events to Cloud Run.
Cloud Scheduler	Managed cron service in Google Cloud.	Can trigger Cloud Run on a schedule.
OIDC	OpenID Connect authentication mechanism.	Used by Scheduler or Pub/Sub to authenticate to Cloud Run.
Cloud Run Service	Request-driven Cloud Run workload with an HTTP endpoint.	Use for APIs, web apps, webhooks, and event targets.

Term	Definition	Why it matters
Cloud Run Job	Batch workload that runs to completion and has no HTTP endpoint.	Use for scripts, migrations, ETL, and scheduled batch tasks.
Task	Unit of work inside a Cloud Run Job.	Jobs can run multiple tasks.
Parallelism	Number of job tasks allowed to run at the same time.	Controls batch execution speed and resource pressure.
Cloud Build	Google Cloud service that builds source code into deployable artifacts.	Used when deploying Cloud Run directly from source.
Buildpacks	Tooling that automatically detects language/runtime and builds a container image.	Used by Cloud Build when no Dockerfile is present.
Dockerfile	File describing how to build a container image.	Gives explicit control over image construction.
CPU boost	Cloud Run feature that gives extra CPU during instance startup.	Helps reduce startup latency.

3. Structured summary of the content

The PDF follows a logical Cloud Run learning path:

Section	Role in the tutorial
Exam relevance	Explains why Cloud Run matters for ACE.
What is Cloud Run?	Defines Cloud Run as a fully managed platform for stateless containers.
Container contract	Explains the rules your container must follow: port, bind address, startup time, statelessness.
Deploying a service	Shows deployment from container image, from source, and via console.
Key deployment parameters	Lists practical knobs: CPU, memory, port, timeout, concurrency, min/max instances, IAM, secrets, VPC.
Revisions and traffic	Explains immutable revisions, rollback, canary releases, and tags.
Authentication and access	Explains public/private access and key IAM roles.
Scaling behavior	Explains scale-to-zero, cold starts, instance limits, and concurrency.
Networking	Explains VPC egress and ingress restrictions.
Secrets and config	Shows secrets as env vars or files.
Event triggers	Shows Pub/Sub, Eventarc, and Cloud Scheduler patterns.
Jobs versus services	Distinguishes HTTP services from batch jobs.
Common operations	Shows update, describe, get URL, and delete commands.
Exam traps	Maps ACE keywords to correct Cloud Run answers.

4. The actual tutorial, on 2 levels

4.1 What Cloud Run is

Level 1 — Easy explanation

Cloud Run is like saying:

> “Here is my container. Please run it when someone needs it, scale it when there is traffic, stop it when there is no traffic, and give me an HTTPS URL.”

You do not create VMs. You do not manage Kubernetes nodes. You do not patch servers. You give Google Cloud a container image or source code, and Cloud Run handles the runtime platform.

Level 2 — Technical explanation

Cloud Run runs containerized workloads on Google-managed infrastructure. For a** ** **Cloud Run Service** , Cloud Run exposes an HTTPS endpoint, receives requests, creates or reuses container instances, routes requests to them, and automatically scales instance count based on demand. Google's documentation describes Cloud Run as fully managed and serverless, meaning infrastructure management is abstracted away. (Google Cloud Documentation)

The PDF highlights that Cloud Run can scale from zero to many instances and supports HTTP/1, HTTP/2, gRPC, and WebSockets.

Why it matters

For the ACE exam, when you see:

> fully managed + container + scale to zero

the expected answer is usually** ** **Cloud Run** , not GKE or Compute Engine.

Where it is used in practice

Cloud Run is used for:

- REST APIs
- web applications
- webhooks
- lightweight microservices
- event handlers
- scheduled HTTP tasks
- internal backend services
- containerized apps that do not need Kubernetes node control

---

4.2 Cloud Run Service versus Cloud Run

Job

Level 1 — Easy explanation

A** **Cloud Run Service** waits for requests.

A** **Cloud Run Job** runs a task and finishes.

Think of it like this:

- Service = restaurant open for customers.
- Job = cleaning crew that comes in, does the work, and leaves.

Level 2 — Technical explanation

A Cloud Run Service has an HTTP endpoint and is request-driven. It is suitable for workloads that must respond to incoming HTTP requests.

A Cloud Run Job has no HTTP endpoint. It executes one or more tasks and exits when work is complete. Google's documentation says a job runs its tasks and exits, unlike a service that listens for and serves requests. (Google Cloud Documentation)

The PDF says Cloud Run Jobs support batch-style execution, parallel tasks, and task timeouts.

Why it matters

ACE questions often phrase this as:

> "Run a script that finishes in 30 minutes."

That is usually** ** **Cloud Run Job** , not Cloud Run Service.

Where it is used in practice

Use Cloud Run Jobs for:

- database migrations
- batch imports
- scheduled reports
- cleanup scripts

- ETL tasks
- parallel processing

Use Cloud Run Services for:

- APIs
- websites
- event endpoints
- webhooks
- request/response workloads

4.3 The Cloud Run container contract

Level 1 — Easy explanation

Cloud Run can only run your container correctly if your app behaves the way Cloud Run expects.

Your container must:

1. Listen on the correct port.
2. Listen on the correct network address.
3. Start quickly enough.
4. Not depend on local files being permanent.

Level 2 — Technical explanation

The PDF lists the core contract:

- listen on the port provided by the** `PORT` **environment variable, default** **`8080`**
- bind to** `0.0.0.0`**, not** **`127.0.0.1`**
- start within the allowed startup window
- remain stateless because local disk is ephemeral

Google's Cloud Run container runtime contract documentation defines the requirements and runtime behaviors for containers running on Cloud Run. (Google Cloud Documentation)

The most common beginner error is binding the app to** `localhost` **/** **`127.0.0.1`**. That means the app is only reachable from inside the container, not from Cloud Run's request

routing layer.

Why it matters

If the container does not listen correctly, Cloud Run may deploy the revision but fail health/startup checks or fail to serve traffic.

Where it is used in practice

In Node.js, Python Flask/FastAPI, Java Spring Boot, Go, or .NET apps, you must configure the server to listen on:

```
host = 0.0.0.0
port = $PORT
```

Not:

```
host = 127.0.0.1
port = 5000
```

unless Cloud Run has been configured to expect port** **5000.

4.4 Deploying from a container image

Code snippet

```
gcloud run deploy my-service \
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \
  --region=us-central1 \
  --platform=managed \
  --allow-unauthenticated \
  --port=8080 \
  --memory=512Mi \
  --cpu=1 \
  --min-instances=0 \
  --max-instances=100 \
  --timeout=300 \
  --concurrency=80 \
```

```
    --set-env-vars="ENV=prod,DB_HOST=10.0.0.5" \
    --service-account=my-sa@PROJECT_ID.iam.gserviceaccount.com
```

The PDF uses this command to show deployment from a container image in Artifact Registry. Google's current docs also describe deploying container images to a new service or a new revision of an existing Cloud Run service. (Google Cloud Documentation)

What it does overall

This command creates or updates a Cloud Run service called** **`my-service`. Cloud Run pulls the specified container image, creates a new immutable revision, configures runtime settings, and exposes the service according to the chosen access and ingress settings.

Line-by-line explanation

Line	Meaning	Why it is there	If missing or wrong
`gcloud run deploy my-service \`	Deploys a Cloud Run service named `my-service`.	Identifies the service to create/update.	A wrong service name deploys a different service.
`--image=...my-app:v1 \`	Tells Cloud Run which container image to run.	Required when deploying from an existing image.	Deployment fails if the image is missing, private without permission, or invalid.
`--region=us-central1 \`	Chooses the region.	Cloud Run services are regional.	You may deploy in the wrong region or be prompted interactively.

Line	Meaning	Why it is there	If missing or wrong
`--platform=managed \`	Uses fully managed Cloud Run.	Matches the exam default.	Usually optional in modern commands, but clarifies managed Cloud Run.
`--allow-unauthenticated \`	Makes the service public.	Useful for public APIs/websites.	Without it, callers need IAM permission and an identity token.
`--port=8080 \`	Configures the container port.	Must match the port the app listens on.	Requests fail if your app listens on a different port.
`--memory=512Mi \`	Gives each instance 512 MiB memory.	Controls memory available to the container.	Too low causes crashes or OOM errors.
`--cpu=1 \`	Gives each instance 1 vCPU.	Controls compute capacity.	Too low may cause slow responses; too high costs more.
`--min-instances=0 \`	Allows scale-to-zero.	Cheapest idle behavior.	Cold starts can occur after idle periods.
`--max-instances=100 \`	Caps scaling at 100 instances.	Protects downstream services.	Too low causes throttling; too high can

Line	Meaning	Why it is there	If missing or wrong
			overload databases.
`--timeout=300 \`	Sets max request duration to 300 seconds.	Allows requests up to five minutes.	Long requests may be killed if timeout is too short.
`--concurrency=80 \`	Allows up to 80 simultaneous requests per instance.	Reduces instance count for concurrent workloads.	Too high can overload one instance; too low may create many instances.
`--set-env-vars="ENV=prod,DB_HOST=10.0.0.5" \`	Injects plain environment variables.	Passes non-secret config to the app.	Missing config can break app startup or connectivity.
`--service-account=...`	Sets runtime identity.	Controls permissions when the service calls GCP APIs.	Default Compute SA may be overprivileged or lack required permissions.

4.5 Deploying from source

Code snippet

```
gcloud run deploy my-service \
  --source=. \
  --region=us-central1 \
  --allow-unauthenticated
```

The PDF says Cloud Build builds the image automatically from source and uses Buildpacks if there is no Dockerfile. Google also documents deploying services from source code with** **`gcloud run deploy --source`. (Google Cloud Documentation)

What it does overall

This command deploys the current source directory to Cloud Run. Instead of you building and pushing the image manually, Cloud Build builds it, stores it, and Cloud Run deploys it.

Line-by-line explanation

Line	Meaning	Why it is there	If missing or wrong
`gcloud run deploy my-service \`	Creates or updates the service.	Names the target Cloud Run service.	Wrong name creates/updates the wrong service.
`--source=. \`	Uses the current directory as source code.	Tells Cloud Build what to build.	Without `--source` or `--image`, Cloud Run does not know what to deploy.
`--region=us-central1 \`	Deploys to the selected region.	Keeps service regional and close to users/resources.	Wrong region can add latency or complicate networking.
`--allow-unauthenticated`	Makes the service publicly callable.	Useful for public endpoints.	Without it, IAM authentication is required.

4.6 Deployment parameters

Level 1 — Easy explanation

Deployment parameters are the knobs you use to tell Cloud Run:

- how much CPU and memory the container gets
- how many requests each instance can handle
- whether the service is public or private
- how many instances it can create
- what secrets and environment variables it receives
- whether it can connect to private networks

Level 2 — Technical explanation

Important parameters from the PDF include:

Parameter	Technical role
`--image`	Container image source.
`--region`	Regional deployment location.
`--port`	Container listening port.
`--memory`	Memory allocated per instance.
`--cpu`	vCPU allocated per instance.
`--min-instances`	Minimum warm instances.
`--max-instances`	Maximum autoscaled instances.
`--concurrency`	Max concurrent requests per instance.
`--timeout`	Max request duration.
`--allow-unauthenticated`	Grants public invocation.
`--ingress`	Restricts network ingress path.
`--service-account`	Runtime identity.
`--set-env-vars`	Plain configuration injection.
`--set-secrets`	Secret Manager integration.
`--vpc-connector`	Serverless VPC Access connector.
`--vpc-egress`	Controls which outbound traffic goes through VPC.
`--cpu-boost`	Improves startup performance.

The PDF lists these parameters and defaults as key exam material.

Why it matters

In real deployments, most Cloud Run production issues come from incorrect values here:

- wrong port
- insufficient memory
- too much concurrency
- missing service account permissions
- public service accidentally exposed
- no VPC path to private database
- wrong min/max instance settings

4.7 Revisions and traffic management

Level 1 — Easy explanation

Every time you deploy, Cloud Run keeps the old version and creates a new version. These versions are called** ** **revisions** .

You can say:

- “Send 100% traffic to the new version.”
- “Send only 10% to the new version.”
- “Rollback to the previous version.”
- “Give this version a preview URL.”

Level 2 — Technical explanation

A Cloud Run revision is an immutable snapshot of the service configuration and container image. Google’s docs explain that Cloud Run lets you specify which revisions receive traffic and what percentages they receive, enabling rollbacks, gradual rollouts, and traffic splitting. (Google Cloud Documentation)

The PDF says every deployment creates a new immutable revision and that revisions remain available for rollback or traffic splitting.

Code: list revisions

```
gcloud run revisions list --service=my-service --region=us-central1
```

Part	Meaning
`gcloud run revisions list`	Lists revisions.
`--service=my-service`	Shows revisions for this service.
`--region=us-central1`	Looks in this region.

If the region or service is wrong, you may see no revisions or the wrong service history.

Code: send 100% traffic to one revision

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00005-abc=100
```

Line	Meaning
`gcloud run services update-traffic my-service \`	Modifies traffic routing for `my-service`.
`--region=us-central1 \`	Applies the change in the selected region.
`--to-revisions=my-service-00005-abc=100`	Sends 100% of traffic to revision `my-service-00005-abc`.

If the revision name is wrong, the command fails. If you send 100% to a broken revision, production traffic breaks.

Code: canary deployment

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00004-xyz=90,my-service-00005-abc=10
```

Line	Meaning
`update-traffic`	Changes request routing.

Line	Meaning
`my-service-00004-xyz=90`	Keeps 90% of traffic on the old revision.
`my-service-00005-abc=10`	Sends 10% of traffic to the new revision.

This is a canary rollout. You test the new revision with a small slice of real traffic before full rollout.

Code: rollback

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=my-service-00004-xyz=100
```

This sends all traffic back to the previous revision.

Code: tag a revision

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --set-tags=preview=my-service-00005-abc
```

This creates a named route to a revision, useful for testing. The PDF gives an example preview URL for a tagged revision.

Why it matters

This is how Cloud Run gives you safe deployment patterns without managing load balancers yourself.

Where it is used in practice

- Canary release: 5% or 10% new version.
- Blue/green style release: switch traffic from old to new.
- Emergency rollback: send traffic back to stable revision.
- Preview testing: tag a revision and test via separate URL.

4.8 Authentication and access control

Level 1 — Easy explanation

Cloud Run services can be:

- public: anyone can call them
- private: only authorized identities can call them

Public access is like leaving the front door open. Private access means callers need a valid badge.

Level 2 — Technical explanation

Cloud Run invocation is controlled through IAM. Public access commonly grants** `allUsers` **the `roles/run.invoker` role. Private access requires authenticated callers with** `roles/run.invoker`.

The PDF identifies these roles:

- `roles/run.invoker` — call the service
- `roles/run.developer` — deploy and manage
- `roles/run.admin` — full control

Google's Cloud Run IAM documentation lists predefined roles for controlling access to Cloud Run resources. (Google Cloud Documentation)

Code: call an authenticated Cloud Run service

```
curl -H "Authorization: Bearer $(gcloud auth print-identity-token)" \
  https://my-service-xxxxx-uc.a.run.app
```

Part	Meaning
`curl`	Sends an HTTP request.
`-H`	Adds an HTTP header.
`Authorization: Bearer ...`	Sends a bearer token proving caller identity.

Part	Meaning
`gcloud auth print-identity-token`	Generates an identity token for the current authenticated principal.
`https://my-service-...run.app`	Cloud Run service URL.

If the caller does not have** **`roles/run.invoker`, the request is denied.

Why it matters

This is an exam trap:

> "Stop public access to Cloud Run."

Correct idea:

```
--no-allow-unauthenticated
```

plus IAM permissions for specific callers.

4.9 Scaling behavior

Level 1 — Easy explanation

Cloud Run automatically adds more container instances when traffic grows and removes them when traffic disappears.

The main knobs are:

- `min-instances`: how many are always ready
- `max-instances`: maximum allowed
- `concurrency`: how many requests one instance handles at once

Level 2 — Technical explanation

The PDF summarizes the important scaling behavior:

- `min-instances=0` allows scale-to-zero and lowest idle cost, but first request may see cold start.
- `min-instances=1+` keeps instances warm and reduces cold starts, but costs money while idle.
- `max-instances` prevents unlimited scaling and protects downstream systems.
- High concurrency means fewer instances are needed.
- `concurrency=1` behaves more like a traditional one-request-per-server model.

Why it matters

The ACE keyword is:

> "Eliminate cold starts."

Expected answer:

```
--min-instances >= 1
```

Where it is used in practice

Scenario	Recommended setting idea
Cost-sensitive dev service	`min-instances=0`
Public API with latency sensitivity	`min-instances=1` or more
Database cannot handle many connections	lower `max-instances`
CPU-heavy app	lower concurrency
I/O-heavy app	higher concurrency may be acceptable

4.10 Networking: ingress and egress

Level 1 — Easy explanation

Cloud Run has two network directions:

- **Ingress** : who can call your Cloud Run service.

- **Egress** : where your Cloud Run service can call out.

Ingress protects the front door. Egress controls the service's outbound path to databases, APIs, and private systems.

Level 2 — Technical explanation

The PDF lists ingress options:

Ingress mode	Meaning
`all`	Open to internet paths.
`internal`	Internal sources only.
`internal-and-cloud-load-balancing`	Access through internal paths and supported Cloud Load Balancing paths.

The PDF also explains VPC egress options, including Direct VPC egress and Serverless VPC Access connectors. Google's current documentation identifies Direct VPC egress as a way for Cloud Run services and jobs to send traffic to a VPC network without a Serverless VPC Access connector, and the comparison page describes Direct VPC egress as the recommended option. (Google Cloud Documentation)

Code: VPC connector egress

```
gcloud run deploy my-service \
  --image=... \
  --vpc-connector=my-connector \
  --vpc-egress=all-traffic
```

Line	Meaning
`gcloud run deploy my-service \`	Deploys or updates the service.
`--image=... \`	Uses the specified container image.
`--vpc-connector=my-connector \`	Routes eligible outbound traffic through a Serverless VPC Access connector.
`--vpc-egress=all-traffic`	Sends all outbound traffic through the VPC connector, not only private ranges.

If the connector is missing, misconfigured, in the wrong region, or lacks firewall routes, private connectivity may fail.

Why it matters

For exam questions like:

> “Cloud Run must connect to Cloud SQL using private IP.”

You need a private networking path: Direct VPC egress or Serverless VPC Access connector, plus correct IAM and database/network configuration.

4.11 Secrets and configuration

Level 1 — Easy explanation

Do not put passwords directly in your container image or source code.

Use:

- environment variables for normal config
- Secret Manager for passwords and sensitive values

Level 2 — Technical explanation

Cloud Run can inject Secret Manager secrets either as environment variables or mounted files. The PDF says the service’s runtime service account must have**
**`roles/secretmanager.secretAccessor`.

Code: secret as environment variable

```
gcloud run deploy my-service \
  --image=... \
  --set-secrets=DB_PASSWORD=db-password:latest
```

Line	Meaning
`gcloud run deploy my-service \`	Deploys or updates the service.
`--image=... \`	Uses the specified image.
`--set-secrets=DB_PASSWORD=db-password:latest`	Injects Secret Manager secret `db-password` version `latest` into env var `DB_PASSWORD`.

If the service account lacks secret access, the service cannot read the secret.

Code: secret as file

```
gcloud run deploy my-service \
  --image=... \
  --set-secrets=/etc/secrets/db=db-password:latest
```

Line	Meaning
`--set-secrets=/etc/secrets/db=db-password:latest`	Mounts the secret value as a file at `/etc/secrets/db`.

Use this when your app expects a credential file, certificate, or config file.

4.12 Triggering Cloud Run from events

Level 1 — Easy explanation

Cloud Run does not only serve browser/API traffic. It can also be triggered by other Google Cloud services.

Common triggers:

- Pub/Sub message
- Eventarc event
- Cloud Scheduler cron job

Level 2 — Technical explanation

Cloud Run can receive events as HTTP requests. Pub/Sub push subscriptions send HTTP POST requests to the Cloud Run URL. Cloud Scheduler can call a Cloud Run endpoint on a schedule. Eventarc is the recommended event routing mechanism in the PDF.

Code: Pub/Sub push subscription

```
gcloud pubsub subscriptions create my-sub \
  --topic=my-topic \
  --push-endpoint=https://my-service-xxxxx-uc.a.run.app/ \
  --push-auth-service-account=invoker-sa@PROJECT_ID.iam.gserviceaccount.com
```

Line	Meaning
`gcloud pubsub subscriptions create my-sub \`	Creates a Pub/Sub subscription named `my-sub`.
`--topic=my-topic \`	Subscribes to messages from topic `my-topic`.
`--push-endpoint=... \`	Sends messages to the Cloud Run URL.
`--push-auth-service-account=...`	Uses this service account to authenticate push requests.

If the service account does not have** **`roles/run.invoker` on the Cloud Run service, Pub/Sub push requests fail.

Code: Cloud Scheduler cron trigger

```
gcloud scheduler jobs create http nightly-job \
  --schedule="0 2 * * *" \
  --uri=https://my-service-xxxxx-uc.a.run.app/run \
  --oidc-service-account-email=scheduler-sa@PROJECT_ID.iam.gserviceaccount.com
```

Line	Meaning
`gcloud scheduler jobs create http nightly-job \`	Creates an HTTP Scheduler job named `nightly-job`.
`--schedule="0 2 * * *" \`	Runs daily at 02:00 according to the configured timezone.
`--uri=.../run \`	Calls this Cloud Run endpoint.

Line	Meaning
`--oidc-service-account-email=...`	Uses OIDC authentication from the Scheduler service account.

If the endpoint is private and the Scheduler service account lacks** **`run.invoker`, the call fails.

4.13 Cloud Run Jobs

Level 1 — Easy explanation

A Cloud Run Job is for work that finishes.

Example:

> "Run this batch script with 10 tasks, but only 5 at a time."

Level 2 — Technical explanation

Cloud Run Jobs execute container tasks to completion. Google's docs say a job does not listen for or serve requests, unlike a Cloud Run service. (Google Cloud Documentation) The PDF states that jobs support batch workloads, parallel tasks, and task timeout configuration.

Code: create a job

```
gcloud run jobs create my-job \
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-batch:v1 \
  --region=us-central1 \
  --tasks=10 \
  --parallelism=5 \
  --task-timeout=3600
```

Line	Meaning
`gcloud run jobs create my-job \`	Creates a Cloud Run Job named `my-job`.
`--image=...my-batch:v1 \`	Uses the batch container image.
`--region=us-central1 \`	Creates the job in the region.

Line	Meaning
`--tasks=10 \`	Runs 10 total task instances.
`--parallelism=5 \`	Runs up to 5 tasks at the same time.
`--task-timeout=3600`	Allows each task to run up to 3600 seconds.

Code: execute the job

```
gcloud run jobs execute my-job --region=us-central1
```

Part	Meaning
`gcloud run jobs execute`	Starts a job execution.
`my-job`	Job to run.
`--region=us-central1`	Region where the job exists.

4.14 Common Cloud Run operations

Update env vars without full redeploy command

```
gcloud run services update my-service \
  --region=us-central1 \
  --update-env-vars=NEW_FLAG=true
```

This updates service configuration and creates a new revision.

Line	Meaning
`gcloud run services update my-service \`	Updates service configuration.
`--region=us-central1 \`	Selects region.
`--update-env-vars=NEW_FLAG=true`	Adds or changes `NEW_FLAG`.

Describe service

```
gcloud run services describe my-service --region=us-central1
```

Shows service details such as URL, latest revision, traffic split, configuration, and status.

Get service URL

```
gcloud run services describe my-service --region=us-central1 \
  --format="value(status.url)"
```

Part	Meaning
`describe`	Fetches service metadata.
`--format="value(status.url)"`	Prints only the service URL.

Delete service

```
gcloud run services delete my-service --region=us-central1
```

Deletes the Cloud Run service in that region.

---

5. Practical platform steps

Step 1: Prepare your container

- **What you do:** Build an app that listens on** **`$PORT`.
- **Where:** In your application code.
- **Why:** Cloud Run injects the expected port.
- **Expected result:** The app starts and listens on** **`0.0.0.0:$PORT`.

Example conceptual app behavior:

```
listen host: 0.0.0.0
listen port: process.env.PORT || 8080
```

Verification:

```
docker run -e PORT=8080 -p 8080:8080 IMAGE
curl http://localhost:8080
```

Step 2: Build and push the image

- **What you do:** Build your container image and push it to Artifact Registry.
- **Where:** Local machine, CI/CD pipeline, or Cloud Build.
- **Why:** Cloud Run needs a container image.
- **Expected result:** Image exists in Artifact Registry.

Example image path:

```
us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1
```

Step 3: Deploy to Cloud Run

Use:

```
gcloud run deploy my-service \
  --image=us-central1-docker.pkg.dev/PROJECT_ID/my-repo/my-app:v1 \
  --region=us-central1 \
  --allow-unauthenticated
```

- **What you do:** Deploy the image.
- **Where:** Cloud Shell, terminal, or CI/CD.
- **Why:** Creates Cloud Run service and revision.
- **Expected result:** Cloud Run returns a service URL.

Verify:

```
curl https://SERVICE_URL
```

Step 4: Configure IAM

For public service:

```
--allow-unauthenticated
```

For private service:

```
--no-allow-unauthenticated
```

Then grant caller:

```
gcloud run services add-iam-policy-binding my-service \
  --region=us-central1 \
  --member=serviceAccount:CALLER_SA@PROJECT_ID.iam.gserviceaccount.com \
  --role=roles/run.invoker
```

- **Expected result:** Only authorized callers can invoke the service.

Step 5: Configure scaling

Example:

```
gcloud run services update my-service \
  --region=us-central1 \
  --min-instances=1 \
  --max-instances=20 \
  --concurrency=40
```

- **Why:** Reduces cold starts and protects downstream systems.
- **Expected result:** At least one warm instance, maximum 20 instances, 40 concurrent requests per instance.

Step 6: Configure secrets

```
gcloud run services update my-service \
  --region=us-central1 \
  --set-secrets=DB_PASSWORD=db-password:latest
```

Also grant the runtime service account:

```
roles/secretmanager.secretAccessor
```

- **Expected result:** App can read** **`DB_PASSWORD`.

Step 7: Configure VPC connectivity if needed

For private resources, use Direct VPC egress or a Serverless VPC Access connector. The PDF mentions both, and Google's current docs describe Direct VPC egress as the recommended option. (Google Cloud Documentation)

Connector example:

```
gcloud run deploy my-service \
  --image=... \
  --vpc-connector=my-connector \
  --vpc-egress=private-ranges-only
```

- **Expected result:** Service can reach private RFC1918/VPC resources.

Step 8: Manage traffic safely

Canary:

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=old-revision=90,new-revision=10
```

Rollback:

```
gcloud run services update-traffic my-service \
  --region=us-central1 \
  --to-revisions=old-revision=100
```

- **Expected result:** Controlled rollout or rollback without redeploying.

---

6. The mechanism behind it

Under the hood

When you deploy to Cloud Run:

1. You provide a container image or source code.
2. If source code is used, Cloud Build builds a container image.
3. Cloud Run creates a new immutable revision.
4. Cloud Run assigns configuration to that revision: CPU, memory, env vars, secrets, service account, networking, concurrency.
5. Cloud Run exposes or updates the service URL.
6. Incoming requests arrive at Google's serving infrastructure.
7. Cloud Run routes requests to the active revision based on traffic percentages.
8. If no instance is available, Cloud Run starts one.
9. The request is delivered to the container on** **`$PORT`.
10. Autoscaling adds/removes instances based on demand.
11. IAM and ingress rules decide who can reach the service.
12. The runtime service account decides what the service can access.

Mental model

```
Developer / CI/CD
      |
      | deploy image or source
      v
+------------------+
|   Cloud Run API   |
+------------------+
      |
      | creates immutable revision
      v
+--------------------------+
```

```
| Revision: image + config    |
| CPU, memory, env, secrets   |
| service account, networking |
+-----------------------------+
      |
      | receives traffic percentage
      v
+-----------------------------+
| Cloud Run Service URL       |
| https://...run.app          |
+-----------------------------+
      |
      | incoming HTTPS request
      v
+-----------------------------+
| Cloud Run request router    |
| IAM + ingress + traffic     |
+-----------------------------+
      |
      | forwards to $PORT
      v
+-----------------------------+
| Container instance          |
| app listens on 0.0.0.0:PORT |
+-----------------------------+
      |
      | optional outbound calls
      v
+-----------------------------+
| GCP APIs / VPC / Cloud SQL  |
| Secret Manager / Pub/Sub    |
+-----------------------------+
```

Caption: Cloud Run separates deployment, revision management, request routing, autoscaling, identity, and networking so you can run containers without managing servers.

---

7. Common mistakes and pitfalls

Mistake	Why it is wrong	Correct understanding
App listens on `127.0.0.1`	Cloud Run cannot route external requests into that listener.	Listen on `0.0.0.0`.
App ignores `$PORT`	Cloud Run expects the configured port.	Use `PORT` env var or configure `--port`correctly.
Storing files locally	Local filesystem is ephemeral.	Use Cloud Storage, Cloud SQL, Firestore, Memorystore, etc.

Mistake	Why it is wrong	Correct understanding
Thinking every deploy overwrites old version	Cloud Run creates revisions.	Old revisions can receive traffic again.
Making service public accidentally	`--allow-unauthenticated` grants public invocation.	Use private access and `roles/run.invoker`when needed.
Confusing runtime service account with deployer identity	Runtime SA is what the app uses after it starts.	Deployer creates resources; runtime SA accesses resources.
Setting concurrency too high	One instance may be overloaded.	Tune based on CPU, memory, latency, and downstream limits.
Setting `max-instances` too high	Can overwhelm database connections.	Use max instances to protect dependencies.
Setting `min-instances=0` for latency-sensitive app	Cold starts can happen.	Use `min-instances >= 1`.
Using Service for batch script	Service is request-driven.	Use Cloud Run Job for finite batch work.
Forgetting `run.invoker` for Pub/Sub/Scheduler	Authenticated triggers need permission.	Grant invoker role to the calling service account.
Thinking ingress and IAM are the same	Ingress is network path; IAM is identity permission.	You often need both.

8. When to use and when not to use this solution

Use Cloud Run when

- You have a stateless containerized app.
- You want serverless autoscaling.
- You want scale-to-zero.
- You need an HTTPS endpoint quickly.

- You do not want to manage Kubernetes nodes.
- You have APIs, webhooks, web apps, or event handlers.
- You have batch work suitable for Cloud Run Jobs.
- You want revision-based traffic splitting and rollback.

Do not use Cloud Run when

- You need full Kubernetes control over nodes, DaemonSets, custom networking, or cluster-level scheduling.
- Your app requires durable local disk.
- Your workload is tightly stateful inside the container.
- You need long-running non-HTTP daemon behavior as a service.
- You need specialized VM hardware or OS-level customization.
- You need complex service mesh or Kubernetes-native platform features.

Alternatives

Requirement	Better option
Full Kubernetes control	GKE
VM-level customization	Compute Engine
Simple function-style code	Cloud Run functions / Cloud Functions-style deployment
Long-running batch container	Cloud Run Job
Data pipeline orchestration	Cloud Composer / Workflows / Dataflow, depending on case
Static website only	Cloud Storage + Cloud CDN or Firebase Hosting

9. ACE exam traps and keywords

If the question says...	Think...
“Fully managed container platform”	Cloud Run
“Scale to zero”	Cloud Run

If the question says...	Think...
“Avoid cold starts”	`--min-instances >= 1`
“Canary deployment”	Revisions + traffic split
“Rollback to previous version”	Send traffic to older revision
“Private service only”	`--no-allow-unauthenticated` + IAM
“Allow only specific caller”	Grant `roles/run.invoker`
“Call private IP resource”	Direct VPC egress or VPC connector
“Run script to completion”	Cloud Run Job
“Cron invoke service”	Cloud Scheduler HTTP job with OIDC
“GCP event triggers container”	Eventarc to Cloud Run
“Pub/Sub calls HTTP endpoint”	Pub/Sub push subscription

10. Final summary

Cloud Run is best understood as a** ** **serverless container runtime** : you give Google Cloud a container, and Cloud Run handles HTTPS serving, autoscaling, revisions, traffic routing, IAM integration, and infrastructure.

The most important things to remember:

- Your container must follow the** ** **container contract** : listen on** **`$PORT`**, bind to **`0.0.0.0`, start quickly, and remain stateless.
- A** **Cloud Run Service** handles HTTP requests.
- A** **Cloud Run Job** runs batch work to completion.
- Every deploy creates an immutable** ** **revision** .
- Traffic can be split between revisions for canaries and rollbacks.
- Public access uses** **`--allow-unauthenticated`**; private access requires IAM ****`roles/run.invoker`.
- `min-instances=0` saves money but allows cold starts.
- `min-instances >= 1` keeps instances warm.
- `max-instances` protects downstream systems.
- `concurrency` controls how many simultaneous requests one instance handles.
- VPC egress is needed when Cloud Run must reach private resources.
- Secrets should come from Secret Manager, not hardcoded config.

The core mental model is:

```
Container image/source
        ↓
Cloud Run deploy
        ↓
Immutable revision
        ↓
Traffic routing
        ↓
Autoscaled container instances
        ↓
Requests, events, or jobs execute
```

For the exam, Cloud Run is the answer whenever the workload is a** ** **stateless container** , should be** ** **fully managed** , can** ** **scale automatically** , and may need to** ** **scale to zero** .